Universidad Nacional de Educación a Distancia
(UNED)

Máster Universitario en Investigación en Inteligencia
Artificial

MEMORIA DE ACTIVIDAD

Grafos RDF y Taxonomías RDFS

Asignatura:

Web Semántica y Enlazado de Datos

Autor
Fernando Martínez Marco

Fecha
2 de febrero de 2026

Curso Académico 2025/2026

Índice

1. Introducción **2**

2. Experimentación con Grafos RDF **2**

3. Diseño de Taxonomías RDFS **3**

4. Análisis de Consultas SPARQL **4**

5. Resultados de la Ejecución **5**

6. Validación en GraphDB **5**
6.1. Exploración de la Taxonomía 5
6.2. Validación del Grafo de Experimentación 6

7. Conclusiones **7**

8. Anexos: Código y Datos **8**
8.1. Script de Implementación (solution.py) 8
8.2. Grafo de Experimentación (graph_experimentation.ttl) 10
8.3. Taxonomía RDFS (graph_rdfs_taxonomy.ttl) 11

1. Introducción

En este trabajo hemos abordado el diseño y la manipulación de grafos RDF con el fin de profundizar en la representación de datos dentro de la Web Semántica. Nuestro objetivo principal fue explorar cómo podemos modelar estructuras de información complejas y cómo emplear RDFS para construir una taxonomía básica que nos facilite el uso de mecanismos de inferencia. Para lograrlo, estructuramos la actividad en dos fases: el desarrollo de un grafo de experimentación con elementos semánticos avanzados y el diseño de una taxonomía universitaria para validar la deducción automática de conocimiento.

2. Experimentación con Grafos RDF

Para la primera parte de la actividad hemos diseñado un grafo de experimentación, disponible en el archivo `graph_experimentation.ttl`. Este modelo que creamos no se limita a una estructura simple de ternas, sino que integra componentes avanzados que enriquecen la descripción semántica de los datos, permitiéndonos obtener una representación más acorde a escenarios de uso reales.

Uno de los puntos que destacamos es el empleo de nodos en blanco (Blank Nodes) para representar la dirección postal de un individuo. Con este enfoque agrupamos propiedades relacionadas, como la calle y el número, bajo una entidad anónima vinculada directamente al sujeto. De esta forma, hemos mantenido la integridad estructural del modelo sin necesidad de asignar identificadores únicos globales a información de carácter local o secundario.

Asimismo, implementamos la técnica de la reificación para permitir que nuestro grafo contenga declaraciones sobre otras declaraciones. En este caso, la hemos utilizado para representar la procedencia de una afirmación, lo que nos permite gestionar la autoría o veracidad de los datos. Consideramos que este mecanismo es fundamental en la Web Semántica para mantener la trazabilidad de la información sin comprometer nuestra base de conocimientos directa.

Para validar nuestro diseño, poblamos el grafo con datos sintéticos de un entorno académico y ejecutamos diversas consultas SPARQL. Los resultados que hemos obtenido confirman que somos capaces de extraer la información de manera precisa, incluso atravesando los niveles de anidamiento de los nodos en blanco que definimos inicialmente.

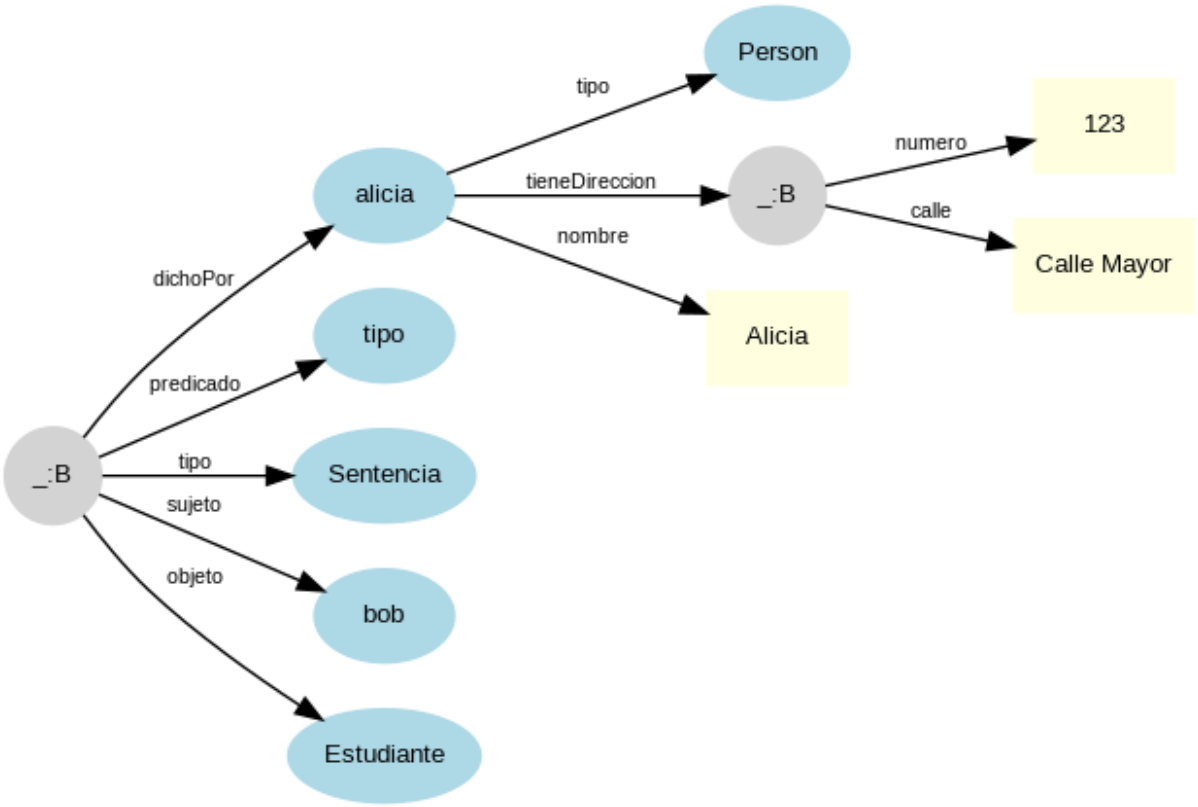

Figura 1: Representacion visual del grafo de experimentacion con entidades en español.

3. Diseño de Taxonomías RDFS

En la segunda fase hemos desarrollado una taxonomía RDFS mínima, recogida en el archivo `graph_rdfs_taxonomy.ttl`. Nuestro propósito fue establecer una jerarquía de clases consistente y definir las restricciones de dominio y rango sobre las propiedades para delimitar el ámbito del conocimiento que queremos representar.

Dentro de este modelo definimos clases base, como la de persona (`Persona`), y subclases especializadas, como la de estudiante (`Estudiante`). Esta estructura jerárquica constituye el pilar de la inferencia en RDFS que hemos aplicado, ya que permite al sistema deducir automáticamente que cualquier instancia de estudiante pertenecerá también a la clase de persona. Además, hemos formalizado propiedades como la de docencia (`enseña`), especificando sus restricciones de dominio y rango, lo que nos permitió derivar el tipo de las entidades involucradas incluso cuando no lo proporcionamos de forma explícita.

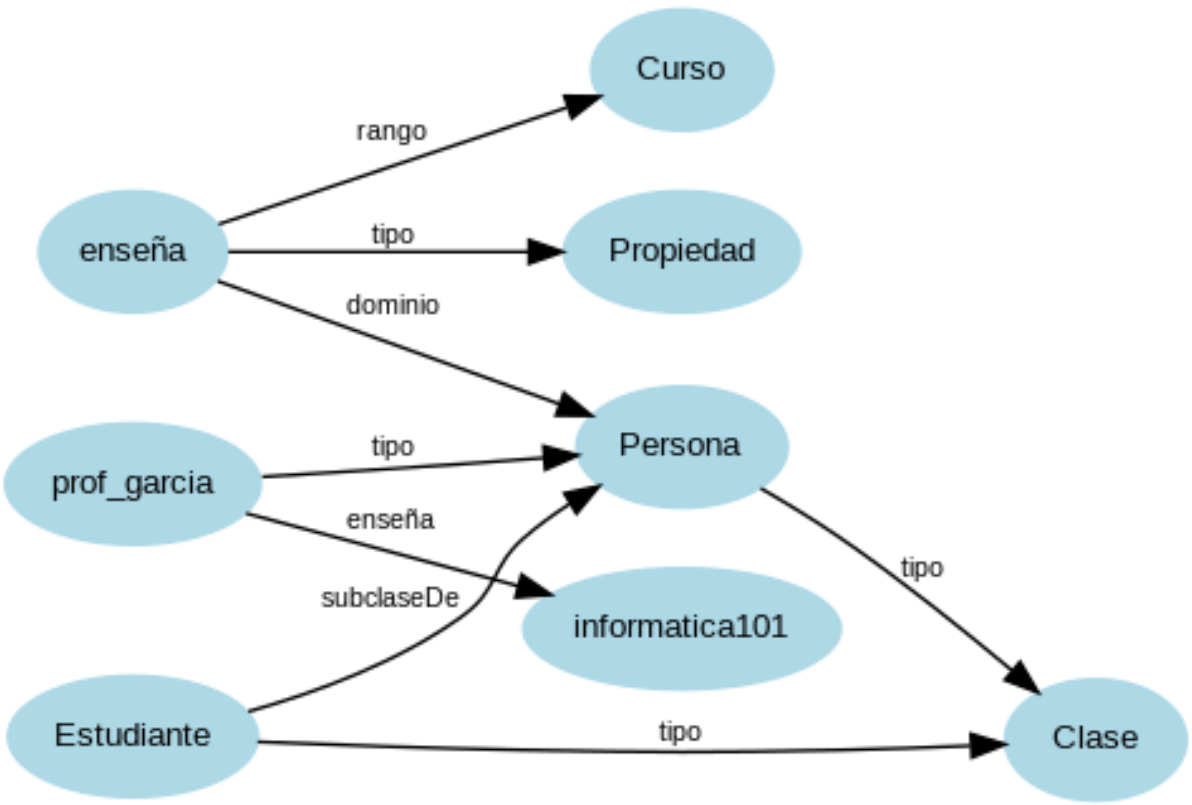

Figura 2: Jerarquia de clases y propiedades con terminos en español.

4. Análisis de Consultas SPARQL

Para validar el correcto funcionamiento de los grafos que diseñamos, hemos implementado consultas en lenguaje SPARQL que nos permiten verificar tanto la extracción de datos como la navegación por estructuras semánticas complejas.

En el grafo de experimentación, diseñamos una consulta orientada a la navegación de nodos en blanco. Buscamos extraer el nombre de una persona y su calle mediante un patrón de triples que conecta al sujeto con un nodo anónimo (`?dir`) y, a partir de ese nodo, extraemos la propiedad de la calle. Con este procedimiento hemos demostrado que, aunque los nodos en blanco carecen de un identificador global, somos capaces de acceder a su contenido de forma programática siempre que exista una relación definida.

En cuanto a la taxonomía RDFS, evaluamos la estructura jerárquica mediante la búsqueda de instancias de la clase `Persona`. Debemos destacar que, en el entorno de consulta estándar sin motor de inferencia activo que hemos usado, SPARQL solo recupera las instancias que declaramos de forma explícita. No obstante, la definición de la taxonomía que realizamos sienta las bases para que un razonador RDFS pueda deducir automáticamente la pertenencia a clases superiores, lo que simplificaría la complejidad de nuestras consultas en modelos de datos a gran escala.

5. Resultados de la Ejecución

Para verificar la integridad de nuestro desarrollo, procedimos a ejecutar el script `solution.py`, el cual realiza la carga de los grafos y la posterior ejecución de las consultas SPARQL que hemos analizado. La salida por consola que obtuvimos es la siguiente:

Salida por consola del script

```
--- Consultas sobre el Grafo de Experimentación ---
Nombre: Alicia, Calle: Calle Mayor

--- Consultas sobre la Taxonomía RDFS ---
Buscando Personas (sin inferencia):
Persona: http://ejemplo.org/prof_garcia
```

Estos resultados nos han confirmado que nuestro sistema es capaz de resolver los patrones de triples, extrayendo datos correctamente tanto de estructuras anidadas como de definiciones explícitas en la taxonomía.

6. Validación en GraphDB

Como complemento a la experimentación programática en Python, hemos procedido a validar la integridad de nuestros modelos en un servidor de grafos especializado: **GraphDB**. Tras importar nuestros archivos `.ttl` en un repositorio local, exploramos visualmente ambos grafos para verificar la robustez del diseño y su compatibilidad con los estándares de la Web Semántica.

6.1. Exploración de la Taxonomía

A través de la herramienta de visualización de GraphDB, generamos una representación expandida de nuestra taxonomía RDFS (`graph_rdfs_taxonomy.ttl`). En la figura 3 comprobamos cómo nuestras entidades personalizadas, como el individuo `prof_garcia`, se integran de forma coherente con el esquema RDFS (`Persona`, `Estudiante`) y, a su vez, cómo estas conectan con los meta-conceptos del estándar del W3C (`Class`, `Property`, `domain`, `range`).

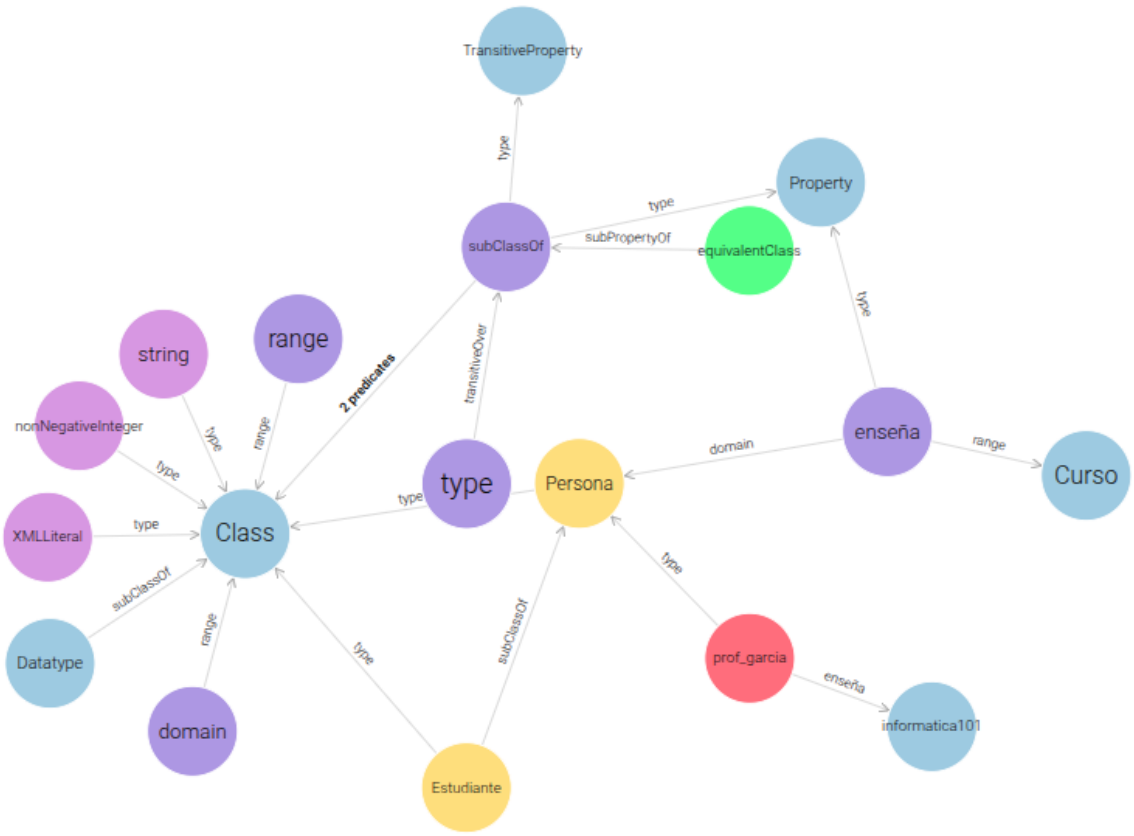

Figura 3: Vista expandida de la taxonomía en GraphDB: integración de datos y meta-conceptos.

6.2. Validación del Grafo de Experimentación

Asimismo, cargamos el grafo de experimentación (`graph_experimentation.ttl`) para verificar la representación de estructuras más complejas como la reificación y los nodos en blanco. Aunque el visualizador de GraphDB aplica ciertas restricciones en la navegación directa de *blank nodes* por su naturaleza anónima, la inspección de los nodos y sus propiedades confirma que la información de direcciones y sentencias reificadas se almacena correctamente siguiendo la lógica de ternas diseñada.

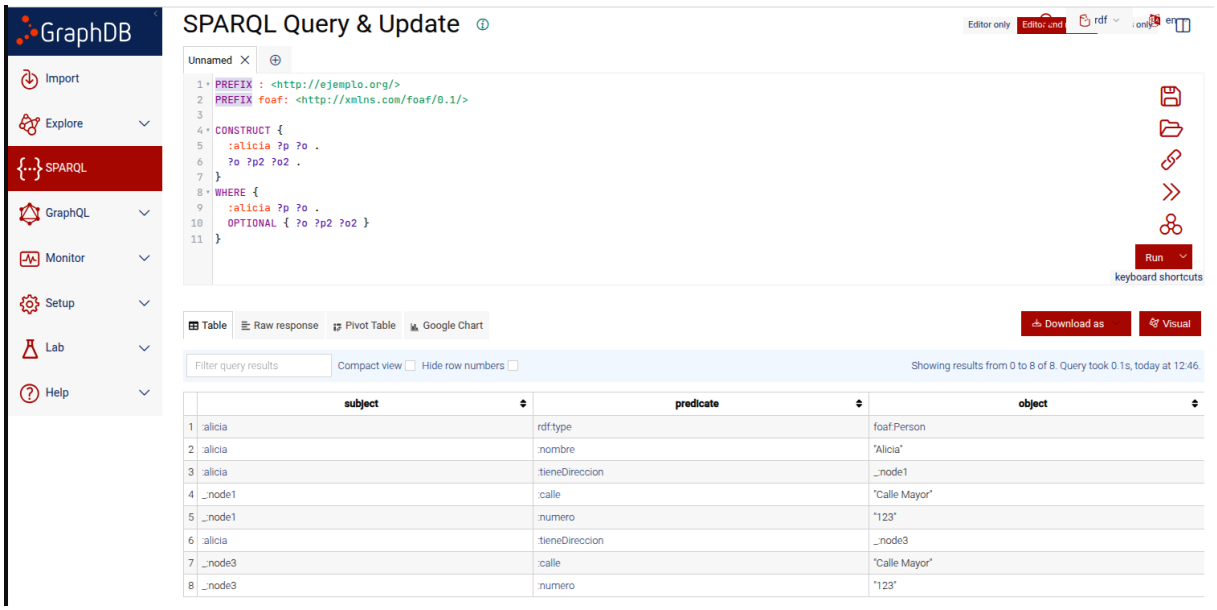

Figura 4: Inspección del grafo de experimentación en la interfaz de GraphDB.

Esta validación confirma que nuestra ontología no solo es funcional a nivel de código, sino que cumple con los requisitos de interoperabilidad en herramientas de gestión de grafos.

7. Conclusiones

Tras realizar este trabajo, hemos comprobado que combinar una buena estructura de grafos con una taxonomía adecuada permite representar el conocimiento de forma rica y escalable. El uso de la librería `rdflib` nos facilitó mucho la gestión de los datos y la ejecución de consultas SPARQL, y la posterior validación en **GraphDB** nos dio la seguridad de estar trabajando sobre modelos totalmente interoperables. En definitiva, esta actividad nos ha servido para entender mejor cómo integrar los datos y la lógica jerárquica en aplicaciones reales de la Web Semántica.

8. Anexos: Código y Datos

8.1. Script de Implementación (solution.py)

A continuación se detalla el código en Python utilizado para la generación y consulta de los grafos RDF empleando la librería `rdflib`.

```python
from rdflib import Graph, Literal, RDF, URIRef, BNode, Namespace
from rdflib.namespace import FOAF, XSD

# 1. Crear el grafo de experimentación
g1 = Graph()
n = Namespace("http://ejemplo.org/")

# Añadir algunos datos
alicia = URIRef("http://ejemplo.org/alicia")
bob = URIRef("http://ejemplo.org/bob")

g1.add((alicia, RDF.type, FOAF.Person))
g1.add((alicia, n.nombre, Literal("Alicia")))

# Uso de un nodo en blanco para la dirección
direccion = BNode()
g1.add((alicia, n.tieneDireccion, direccion))
g1.add((direccion, n.calle, Literal("Calle Mayor")))
g1.add((direccion, n.numero, Literal("123")))

# Ejemplo de Reificación (una sentencia sobre otra sentencia)
# Declaración: "Alicia afirma que Bob es un estudiante"
sentencia = BNode()
g1.add((sentencia, RDF.type, RDF.Statement))
g1.add((sentencia, RDF.subject, bob))
g1.add((sentencia, RDF.predicate, RDF.type))
g1.add((sentencia, RDF.object, n.Estudiante))
g1.add((sentencia, n.dichoPor, alicia))

# Guardar el grafo de experimentación
g1.serialize(destination="graph_experimentation.ttl", format="
    turtle")

# 2. Crear el grafo para la taxonomía RDFS
g2 = Graph()
rdfs = Namespace("http://www.w3.org/2000/01/rdf-schema#")

# Definir clases y jerarquía
g2.add((n.Persona, RDF.type, rdfs.Class))
g2.add((n.Estudiante, RDF.type, rdfs.Class))
g2.add((n.Estudiante, rdfs.subClassOf, n.Persona))

# Definir propiedades, dominio y rango
g2.add((n.enseña, RDF.type, RDF.Property))
g2.add((n.enseña, rdfs.domain, n.Persona))
```

```python
g2.add((n.enseña, rdfs.range, n.Curso))

# Añadir algunas instancias para probar
prof_garcia = URIRef("http://ejemplo.org/prof_garcia")
informatica101 = URIRef("http://ejemplo.org/informatica101")
g2.add((prof_garcia, RDF.type, n.Persona))
g2.add((prof_garcia, n.enseña, informatica101))

# Guardar el grafo de la taxonomía
g2.serialize(destination="graph_rdfs_taxonomy.ttl", format="
    turtle")

# 3. Ejecutar algunas consultas SPARQL de prueba
print("--- Consultas sobre el Grafo de Experimentación ---")
q1 = """
SELECT ?nombre ?calle
WHERE {
    ?p <http://ejemplo.org/nombre> ?nombre .
    ?p <http://ejemplo.org/tieneDireccion> ?dir .
    ?dir <http://ejemplo.org/calle> ?calle .
}
"""
for row in g1.query(q1):
    print(f"Nombre: {row.nombre}, Calle: {row.calle}")

print("\n--- Consultas sobre la Taxonomía RDFS ---")
# Consulta simple: ¿Quiénes son Personas?
q2 = """
SELECT ?p
WHERE {
    ?p a <http://ejemplo.org/Persona> .
}
"""
print("Buscando Personas (sin inferencia):")
for row in g2.query(q2):
    print(f"Persona: {row.p}")
```

8.2. Grafo de Experimentación (graph_experimentation.ttl)

Contenido del grafo generado que incluye nodos en blanco y reificación.

```
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
@prefix ns1: <http://ejemplo.org/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

ns1:alicia a foaf:Person ;
    ns1:nombre "Alicia" ;
    ns1:tieneDireccion [ ns1:calle "Calle Mayor" ;
            ns1:numero "123" ] .

[] a rdf:Statement ;
    ns1:dichoPor ns1:alicia ;
    rdf:object ns1:Estudiante ;
    rdf:predicate rdf:type ;
    rdf:subject ns1:bob .
```

8.3. Taxonomía RDFS (graph_rdfs_taxonomy.ttl)

Definición de las clases y propiedades jerárquicas en formato Turtle.

```
@prefix ns1: <http://ejemplo.org/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

ns1:Estudiante a rdfs:Class ;
    rdfs:subClassOf ns1:Persona .

ns1:Persona a rdfs:Class .

ns1:enseña a rdf:Property ;
    rdfs:domain ns1:Persona ;
    rdfs:range ns1:Curso .

ns1:prof_garcia a ns1:Persona ;
    ns1:enseña ns1:informatica101 .
```